Wi-Fi Controlled Robot Car

using NodeMCU ESP8266 & L298N Motor Driver

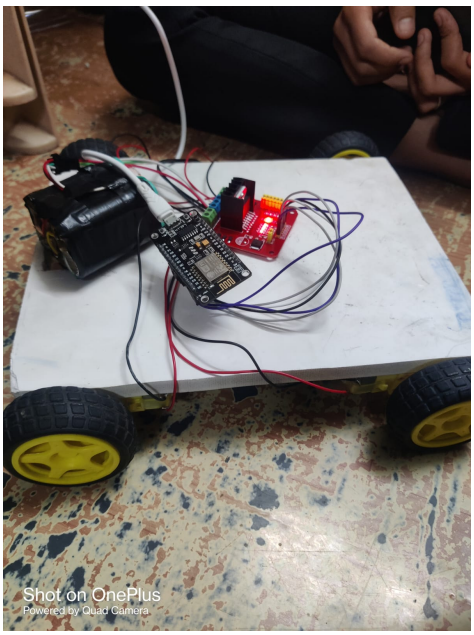

Fig 1: Assembled hardware prototype

Project Report

1. Introduction

This project is a four-wheeled robot car that is controlled wirelessly over Wi-Fi using a NodeMCU ESP8266 microcontroller. The ESP8266 creates its own Wi-Fi access point, hosts a small web server, and drives two DC motor channels through an L298N motor driver module based on simple HTTP commands received from a connected phone or laptop. The car can move forward, backward, turn left, turn right, and stop.

1.1 Objective

To design and build a low-cost Wi-Fi controlled car that can be driven remotely without needing internet access, by using the ESP8266's built-in Access Point (AP) mode and a browser or app sending simple HTTP GET requests.

2. Components Used

S.No	Component	Quantity	Purpose
1	NodeMCU ESP8266	1	Main controller + Wi-Fi module
2	L298N Motor Driver Module	1	Drives the 4 DC motors
3	4WD Robot Car Chassis with DC Motors	1	Mechanical base + 4 geared motors
4	Li-ion Battery Pack (2S, ~7.4V)	1	Power supply
5	Jumper Wires	As required	Connections between modules
6	Micro-USB Cable	1	Programming the NodeMCU

3. Circuit Diagram & Connections

The NodeMCU's GPIO pins are used to control the L298N driver's direction pins. The L298N in turn switches power to the two motor channels (left pair and right pair of wheels), while the battery supplies power to the L298N which also regulates a lower voltage back for the NodeMCU.

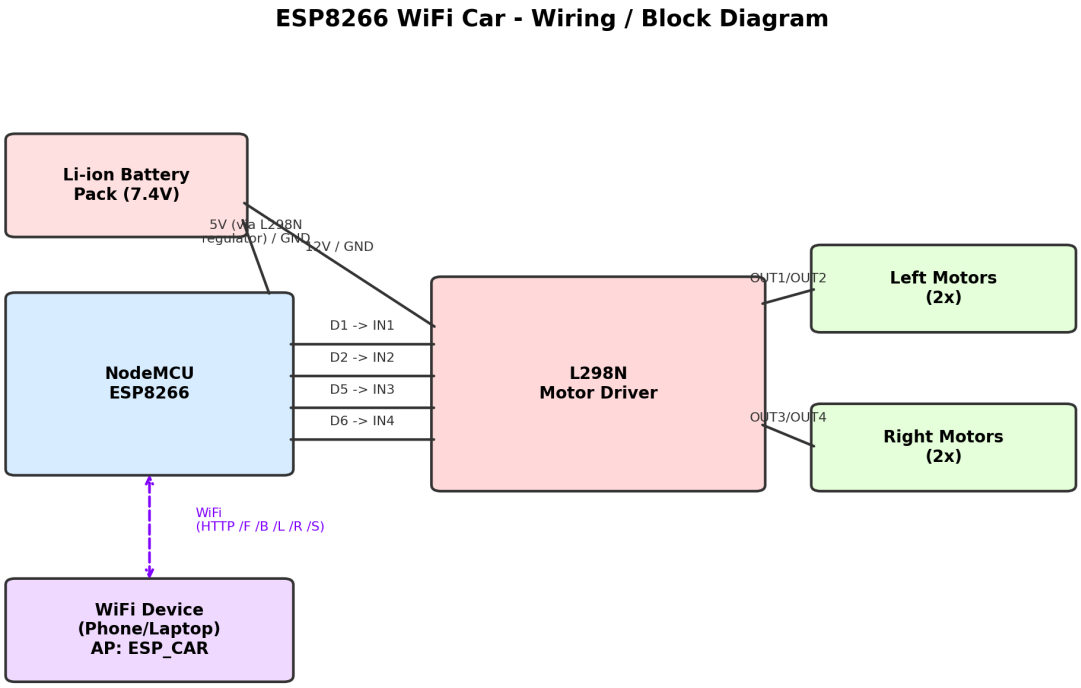

Fig 2: Block wiring diagram of the connections

3.1 Pin Connection Table

NodeMCU Pin	L298N Pin	Function
D1	IN1	Left motor direction control A
D2	IN2	Left motor direction control B
D5	IN3	Right motor direction control A
D6	IN4	Right motor direction control B
VIN / 5V	5V OUT / VCC	Power to NodeMCU (from L298N regulator)
GND	GND	Common ground
-	12V, GND (screw terminal)	Battery pack input
-	OUT1, OUT2	Left side motors
-	OUT3, OUT4	Right side motors

4. Working Principle

On power-up, the NodeMCU starts a Wi-Fi access point named **ESP_CAR** and begins listening for HTTP connections on port 80. A phone or laptop connects to this access point directly (no internet router is needed). When the user sends a request such as **/F**, **/B**, **/L**, **/R**, or **/S** (for example by opening a URL like **http://192.168.4.1/F** in a browser, or through a simple control app/webpage with buttons), the ESP8266 reads the request text and matches it to a movement function.

Each movement function sets the four GPIO pins (D1, D2, D5, D6) HIGH or LOW in different combinations to control the current direction through the L298N's H-bridge circuits, which in turn spins the left and right motor pairs forward or backward:

- Forward: both motor pairs spin in the forward direction.
- Backward: both motor pairs spin in the reverse direction.
- Left: right-side motors spin forward while left-side motors spin backward (pivot turn).
- Right: left-side motors spin forward while right-side motors spin backward (pivot turn).
- Stop: all four control pins are set LOW, cutting motor drive.

5. Source Code (Arduino IDE - ESP8266)

The code below runs on the NodeMCU. It configures the AP, starts the web server, and maps incoming HTTP paths to motor control functions.

```
#include <ESP8266WiFi.h>

const char* ssid = "ESP_CAR";
const char* password = "12345678";

WiFiServer server(80);

#define IN1 D1
#define IN2 D2
#define IN3 D5
#define IN4 D6

void stopCar() {
digitalWrite(IN1, LOW);
digitalWrite(IN2, LOW);
digitalWrite(IN3, LOW);
digitalWrite(IN4, LOW);
}

void forward() {
digitalWrite(IN1, HIGH);
digitalWrite(IN2, LOW);
digitalWrite(IN3, HIGH);
digitalWrite(IN4, LOW);
}

void backward() {
digitalWrite(IN1, LOW);
digitalWrite(IN2, HIGH);
digitalWrite(IN3, LOW);
digitalWrite(IN4, HIGH);
}

void left() {
digitalWrite(IN1, LOW);
digitalWrite(IN2, HIGH);
digitalWrite(IN3, HIGH);
digitalWrite(IN4, LOW);
}
```

```
void right() {
digitalWrite(IN1, HIGH);
digitalWrite(IN2, LOW);
digitalWrite(IN3, LOW);
digitalWrite(IN4, HIGH);
}

void setup() {
pinMode(IN1, OUTPUT);
pinMode(IN2, OUTPUT);
pinMode(IN3, OUTPUT);
pinMode(IN4, OUTPUT);

stopCar();
WiFi.softAP(ssid, password);
server.begin();
}

void loop() {
WiFiClient client = server.available();
if (!client) return;

String req = client.readStringUntil('\r');
client.flush();

if (req.indexOf("/F") != -1) forward();
else if (req.indexOf("/B") != -1) backward();
else if (req.indexOf("/L") != -1) left();
else if (req.indexOf("/R") != -1) right();
else if (req.indexOf("/S") != -1) stopCar();

client.println("HTTP/1.1 200 OK");
client.println("Content-Type:text/html");
client.println();
client.println("<h2>ESP8266 Car</h2>");
}
```

6. Setup & Connection Steps

1. Upload the Arduino sketch above to the NodeMCU ESP8266 using the Arduino IDE (Board: NodeMCU 1.0, correct COM port selected).
2. Wire the NodeMCU, L298N driver, motors, and battery pack exactly as per the pin table in Section 3.1.
3. Power on the car; the NodeMCU will start broadcasting a Wi-Fi network named 'ESP_CAR' with password '12345678'.
4. On a phone or laptop, connect to the 'ESP_CAR' Wi-Fi network.
5. Open a browser (or a simple control webpage/app) and send requests to the car's IP address (default 192.168.4.1), e.g. http://192.168.4.1/F to move forward.
6. Use /F, /B, /L, /R and /S endpoints to move forward, backward, turn left, turn right, and stop respectively.

7. Conclusion

This project successfully demonstrates a simple and low-cost Wi-Fi controlled robot car built using a NodeMCU ESP8266 and an L298N motor driver. By using the ESP8266's own access point, the car can be controlled directly from a phone or laptop without requiring an internet connection or external router, making it portable and easy to operate. The project shows the core concepts of embedded Wi-Fi communication, motor direction control through an H-bridge driver, and basic web-server based command handling.

The design can be further improved by adding speed control using PWM signals, an ultrasonic sensor for obstacle avoidance, a camera module for live video streaming, or a dedicated mobile app interface in place of manual URL requests. Overall, the project provides a solid practical foundation for learning IoT-based robotics and wireless embedded systems.